HW06 Final Report — Three Full API Testing Pipelines

1. Evidence basis and reporting rules

This report covers three selected mutation APIs as three complete pipelines:

1. Registration: `POST /api/register`.
2. Coupon application: `POST /api/apply-coupon`.
3. Product creation: `POST /api/products`.

All design counts come from `23127334_HW06_API_TestCases.xlsx`. All execution counts come from the retained Newman CLI/HTML reports summarized in `postman-run-summary.md`. The workbook contains the actual status, response excerpt, classification, failed assertions, bug ID, and evidence reference for every row.

The final workbook contains 148 primary cases: 120 AI-generated cases and 28 human-added cases. Newman executed each API with three data iterations after a separate backend/database reset, producing 9 iterations, 960 requests, 3,204 passed assertions, and 374 failed assertions. The 960 requests include setup, supporting, primary, verification, and teardown traffic; they are not presented as 960 distinct test cases.

AI audit verdict and execution verdict are deliberately separate:

- `VALID`, `INVALID`, and `INCOMPLETE` assess the quality of the AI-authored test as written.
- `PASS`, `SUT BUG`, `TEST SCRIPT BUG`, and `ENVIRONMENT/SETUP FAILURE` classify execution.
- A test can execute and observe a response while remaining `INVALID` or `INCOMPLETE`. Those outputs remain visible below and in the workbook; they were not relabelled to improve the result.

2. Selection rationale

The three APIs were selected because together they cover three materially different risk surfaces documented in the contract matrix:

Pipeline	Selection rationale	Principal risk exposed
Register	Public identity-creation mutation with password policy, confirmation, uniqueness, persistence, and credential-storage obligations	Input validation and SEC-01 password storage
Coupon	Authenticated business-rule calculation with five eligibility conditions, inclusive minimum, usage state, percent/fixed formulas, and user identity	Missing authorization, state setup, `>=` boundary, and financial calculation

Pipeline	Selection rationale	Principal risk exposed
Product	Admin-only persistent catalog mutation with role enforcement, field constraints, category reference, retrieval, and cleanup	Authentication/authorization, validation, referential integrity, and persistent side effects

This selection is not three variations of the same CRUD path. Registration is intentionally public but handles credentials; coupon application is a conditional calculation; product creation is an admin-only write. The combination makes the audit distinguish expected public access from missing authorization and pure response defects from persistent side effects.

3. Pipeline 1 — Registration

3.1 Contract analysis

The registration contract maps to FR-01 and SEC-01/SEC-05. The request requires `name`, `email`, `password`, and—according to FR-01—matching `confirm_password`; the API specification omits the confirmation field, so that mismatch was retained as a contract gap rather than silently resolved. A valid registration returns HTTP 200 with exactly a string `message` and positive integer `id`, creates one default-role user, and permits subsequent login. Invalid required fields, email format, password strength, or confirmation must not create a user. SEC-01 requires non-plaintext password storage.

Supporting requests were used only to establish and observe the contract: login checks the new identity, admin user lookup checks persistence, and deletion removes the created user. Current implementation behavior was recorded separately and was not used to rewrite expected results.

3.2 AI generation and human audit

The AI produced 40 Register cases spanning equivalence partitions, password boundaries, relational confirmation checks, malformed/top-level JSON, Unicode, injection-oriented inputs, exact response schema, persistence, login, and plaintext-storage checks.

Workbook audit result: 32 `VALID`, 0 `INVALID`, and 8 `INCOMPLETE`. The eight incomplete outputs were retained explicitly:

Verdict	IDs	Why they were not accepted as written
INCOMPLETE	`REG-AI-002`, `REG-AI-003`	One-character “minimum” and emoji acceptance lacked a stated name policy
INCOMPLETE	`REG-AI-009`, `REG-AI-010`	SQLi/XSS cases forced HTTP 200 or mixed API persistence with unbounded UI execution checks
INCOMPLETE	`REG-AI-018`, `REG-AI-019`	Whitespace normalization and duplicate-email status were not uniquely specified
INCOMPLETE	`REG-AI-038`, `REG-AI-039`	Media-type and unknown-field behavior allowed alternative oracles

All eight Register incomplete cases have a corrected-version entry in the workbook; the original verdict and reasoning remain present. For example, the one-character case was reframed from an unsupported BVA minimum to a current-contract valid-string partition, while the emoji was removed from the Unicode case. The audit source remains `23127334_HW06_AI_Audit.md`, which lists every original verdict instead of presenting only corrected cases.

3.3 Human extension

Register has 9 raw human-added cases. Four were excluded from the qualifying human-extension count because their primary logic overlapped an existing case; none was excluded for using a supporting API as the main objective. Therefore exactly 5 qualify for the human gate. The added coverage includes a no-persistence unauthorized/side-effect chain, media-type and duplicate-key parser behavior, exact composite JSON types, and lifecycle/storage checks documented in the human-added design and decision history.

The workbook Summary records: raw human-added `9`, logic-overlap excluded `4`, supporting-main excluded `0`, counted human-added `5`, and final design gate `PASS`. Exclusion did not delete a row; excluded cases remain available for audit and execution.

3.4 Postman implementation and execution result

The Register folder contains 49 primary cases: 40 AI-generated plus 9 human-added. Newman ran `00 Setup`, `API1 Register`, and `99 Verification-Teardown` with `register-data.json` for 3 iterations. The retained result is:

Primary cases	PASS	SUT bug classification	Test script bug	Environment/setup failure	Requests	Assertions passed	Assertions failed
49	6	36	7	0	318	1,053	135

The Newman process exited `1`; this is expected when audited assertions fail. Evidence: Register CLI, Register HTML, and backend reset log.

The 36 execution rows initially classified as candidate SUT failures were triage input, not 36 verified bugs. In particular, chained DB/UI/storage checks missing from the collection were classified as test defects rather than promoted to product defects.

3.5 Verified bug

- `VB-01 — Registration stores password as plaintext`, severity Critical. Two independent requests used distinct credentials, each after a fresh backend/database reset; SQLite stored the exact submitted password in both trials. Raw request, response, and DB observations are retained at the linked evidence. The published record is in `github-issues.md` and Issue #49.

No other Register candidate was promoted without two independent reproductions and a valid, observable oracle.

4. Pipeline 2 — Coupon application

4.1 Contract analysis

FR-09 requires all five conditions: the coupon exists and is active, is unexpired, satisfies `total_amount >= min_order_amount`, is requested by an authenticated user, and remains below that user's usage limit. The percent formula is `total × discount_value / 100`; fixed discount equals `discount_value`; final amount equals total minus discount. Apply-coupon is a preview and must not itself increment usage.

The contract also exposes real gaps: `user_id` is present in the body although identity should be derived from JWT; auth failures permit an unresolved 401/403 choice; usage-limit failure permits 400/409; expiration equality and fractional currency rounding are unspecified. Those gaps were not used to excuse the independently testable missing-header, inclusive-boundary, or exact integer-percent failures.

4.2 AI generation and human audit

The AI produced 40 Coupon cases using a five-condition decision table, BVA around seeded minimum values, percent and fixed calculations, expiration/usage states, repeated application, identity partitions, JWT negatives, malformed input, schema, and no-mutation checks.

Workbook audit result: 31 `VALID`, 1 `INVALID`, and 8 `INCOMPLETE`. None was hidden:

Verdict	IDs	Audit finding
INVALID	`CPN-AI-016`	Forced exact fractional monetary results despite the acknowledged absence of a rounding policy
INCOMPLETE	`CPN-AI-007`, `CPN-AI-022`	Usage-limit rejection retained 400/409 alternatives
INCOMPLETE	`CPN-AI-019`	Equality at expiration lacked `<` versus `<=` semantics
INCOMPLETE	`CPN-AI-025`, `CPN-AI-026`	Body identity versus JWT identity and mismatch status were not resolved
INCOMPLETE	`CPN-AI-029`, `CPN-AI-030`	Invalid/expired JWT status remained 401/403
INCOMPLETE	`CPN-AI-039`	Unknown-field behavior remained strict-reject versus ignore

The workbook reports `Corrected = 0` for Coupon. The invalid and incomplete rows therefore remain as-written and are not represented as approved replacements. Fractional rounding remains a specification gap; the verified calculation bug instead uses 500,000, for which 10% is exactly 50,000 and requires no rounding decision.

4.3 Human extension

Coupon has 10 raw human-added cases. Two overlap existing primary logic and three use a supporting endpoint as their main objective, so only 5 count toward the selected Coupon API's human gate. The

qualifying additions target direct apply-coupon behavior such as identity binding, raw parser/media-type cases, composite amount types, and state/side-effect chains. Supporting checkout or coupon-administration scenarios remain in the workbook but are not counted as direct apply-coupon extensions.

The workbook Summary records: raw human-added `10`, logic-overlap excluded `2`, supporting-main excluded `3`, counted human-added `5`, and final design gate `PASS`.

4.4 Postman implementation and execution result

The Coupon folder contains 50 primary cases: 40 AI-generated plus 10 human-added. Newman ran `00 Setup`, `API2 Coupon`, and `99 Verification-Teardown` with `coupon-data.json` for 3 iterations:

Primary cases	PASS	SUT bug classification	Test script bug	Environment/setup failure	Requests	Assertions passed	Assertions failed
50	15	20	3	12	324	1,068	122

Evidence: Coupon CLI, Coupon HTML, and backend reset log. The process exited `1` because audited assertions failed.

The 12 environment/setup failures were not called SUT bugs: disabled, usage-limit, concurrency, and expired-token cases lacked a proven fixture or token in that isolated run. Three incomplete chained/state checks were classified as test-script bugs. This separation prevents a response obtained under the wrong precondition from becoming a false defect report.

4.5 Verified bugs

- `VB-02 — Apply-coupon succeeds without Authorization`, severity High. Both independent no-header trials returned HTTP 200 and disclosed coupon calculation, contradicting the authentication requirement regardless of the unresolved 401/403 choice.
- `VB-04 — Coupon rejects the inclusive minimum boundary`, severity High. With `SAVE10.min_order_amount = 300000`, equality returned HTTP 400 in 2/2 reset trials, contradicting explicit `>=` semantics.
- `VB-05 — Percent coupon calculation uses the wrong formula`, severity Critical. For total 500,000 and SAVE10, both trials returned `discount_amount=-4500000` and `final_amount=5000000` instead of 50,000 and 450,000.

Each linked section contains the two raw request/response trials. Published issue records and related FR/SEC fields are retained in `github-issues.md`, with live links to Issues #49–#53.

5. Pipeline 3 — Product creation

5.1 Contract analysis

Product creation maps to FR-12, FR-15, SEC-02, SEC-03, SEC-04, and SEC-05. It requires a valid admin JWT, a non-empty name, positive numeric price, and positive existing `category_id`; description and image URL are optional under the audited requirement. Success returns HTTP 200 with exact `message` and positive integer `id`, creates one retrievable product, and leaves unrelated product/category state unchanged. Guest and non-admin writes must be rejected without persistence.

The contract matrix records alternative statuses where the source is not definitive: malformed/expired tokens may use 401 or 403, and a nonexistent category may use 400 or 422. It also distinguishes downstream safe rendering from what one API test can prove.

5.2 AI generation and human audit

The AI produced 40 Product cases across guest/user/admin authorization partitions, field omission and exact JSON types, name and price boundaries, category references, optional fields, injection-oriented values, mass assignment, malformed JSON, schema, retrieval, and cleanup.

Workbook audit result: 26 `VALID`, 6 `INVALID`, and 8 `INCOMPLETE`. This is the weakest AI set and is reported without suppression:

Verdict	IDs	Audit finding
INVALID	`PRD-AI-013`, `PRD-AI-014`, `PRD-AI-015`	Invented name boundaries at 254/255/256 without a cited requirement in the AI input
INVALID	`PRD-AI-018`	Combined one POST request with proof across “all UI sinks” and forced acceptance
INVALID	`PRD-AI-024`	Forced acceptance of price 0.01 without a currency-precision rule
INVALID	`PRD-AI-026`	Assumed acceptance of 2,147,483,647 merely because no maximum was stated
INCOMPLETE	`PRD-AI-004`, `PRD-AI-005`, `PRD-AI-006`	Malformed, expired, and tampered token status remained 401/403
INCOMPLETE	`PRD-AI-012`, `PRD-AI-016`	One-character and emoji acceptance lacked a stated character policy
INCOMPLETE	`PRD-AI-017`	SQLi test forced HTTP 200 instead of testing safe non-injection under either permitted accept/reject behavior
INCOMPLETE	`PRD-AI-032`, `PRD-AI-037`	Referential-integrity and unknown-field statuses were not uniquely specified

The workbook reports `Corrected = 0` for Product. All six invalid and eight incomplete outputs remain present in the workbook, were not counted as corrected, and were not used as verified-bug oracles where the stated expectation was unsupported.

5.3 Human extension

Product has 9 raw human-added cases. Three overlap existing logic and one uses a supporting endpoint as its main objective, leaving 5 qualifying direct Product extensions. Human cases add persistent unauthorized-write chains, wrong media type, duplicate JSON keys, object/array price partitions, ID-parity schema drift, unsafe URL concerns, referential persistence, and lifecycle deletion. Cases that overlap an AI objective still remain in the design but do not inflate the human gate.

The workbook Summary records: raw human-added `9`, logic-overlap excluded `3`, supporting-main excluded `1`, counted human-added `5`, and final design gate `PASS`.

5.4 Postman implementation and execution result

The Product folder contains 49 primary cases: 40 AI-generated plus 9 human-added. Newman ran `00 Setup`, `API3 Product`, and `99 Verification-Teardown` with `product-data.json` for 3 iterations:

Primary cases	PASS	SUT bug classification	Test script bug	Environment/setup failure	Requests	Assertions passed	Assertions failed
49	10	32	6	1	318	1,083	117

Evidence: Product CLI, Product HTML, and backend reset log. The process exited `1` because audited assertions failed.

The 32 candidate SUT classifications include repeated validation/auth deviations across related cases; they are not 32 distinct verified defects. Six cases lacked the complete chained/UI/storage oracle and were classified as test-script defects; one expired-token case lacked its required environment state.

5.5 Verified bug

- `VB-03 — Product creation succeeds and persists without admin Authorization`, severity Critical. In both independent reset trials, a request with no Authorization header returned HTTP 200 and created a product row. The linked evidence contains the raw requests, responses, distinct markers, and DB observations. The published record is in `github-issues.md` and Issue #51.

6. Cross-pipeline AI generation and human audit result

The AI generation stage produced exactly 40 cases per API. Human audit preserved all outputs and yielded:

API	AI-generated	VALID	INVALID	INCOMPLETE	Corrected	Raw human-added	Qualifying human-added
Register	40	32	0	8	8	9	5
Coupon	40	31	1	8	0	10	5
Product	40	26	6	8	0	9	5
Total	**120**	**89**	**7**	**24**	**8**	**28**	**15**

Thus 31 of 120 AI outputs were not valid as written: 7 invalid and 24 incomplete. They remain identifiable above and row-by-row in the AI audit. The human-extension gate initially failed after overlap/supporting-endpoint exclusions; eight newly reviewed candidates (`REG-H-008`, `REG-H-009`, `CPN-H-008` – `CPN-H-010`, and `PRD-H-007` – `PRD-H-009`) raised each pipeline to exactly five qualifying human cases. This supplementation and the exclusions are retained in `p4-final-design-check.md` and the workbook Summary.

The reusable Agent Skill is demonstrated with exactly one API operation in the real student-recorded YouTube video. The recording is supplementary evidence for the specification-to-candidates workflow, deterministic validation, blocked export before human review, manual approval, and the final export gate; it does not replace the auditable repository artifacts.

7. Postman implementation and features actually used

The executable artifact is `23127334_HW06_API_Testing.postman_collection.json`, with a sanitized public environment template. The implementation used these evidenced features:

- Collection v2.1 with `00 Setup`, three API folders, technique subfolders (`Domain`, `State`, `Security`, `Schema`), and `99 Verification-Teardown`.
- Collection variables for run ID, cleanup ID stacks, and active TC_ID; environment variables for base URL, student ID, runtime credentials/tokens, and captured IDs; request/local variables for data-row values and unique markers.
- A collection pre-request script that upserts and asserts `X-Student-Id: 23127334` and logs URL/header/timestamp. The real Postman Console evidence is `23127334-x-student-id-console-20260817-140106Z.png`.
- Request pre-request scripts that bind TC_ID and deterministic iteration data.
- Test scripts for status, JSON content type, exact schema, business values, sensitive-field absence, persistence, supporting reads, and cleanup.
- Three external data files and three isolated Newman invocations, each with 3 iterations.
- Newman `6.2.1` with CLI, JSON, and htmlextra reporters; public HTML used `--reporter-htmlextra-skipSensitiveData`. Machine JSON containing resolved runtime auth data remained Git-ignored.
- Sanitized environment export and executable collection export.

The Postman GUI Collection Runner is not claimed: no retained evidence proves it was used. Newman is the authoritative automated execution evidence. The full evidence mapping is in `postman-features.md`.

8. Combined execution and bug triage

API	Primary cases	PASS	Candidate SUT failures	Test defects	Environment defects	Requests	Assertions passed	Assertions failed
Register	49	6	36	7	0	318	1,053	135
Coupon	50	15	20	3	12	324	1,068	122
Product	49	10	32	6	1	318	1,083	117
Total	**148**	**31**	**88**	**16**	**13**	**960**	**3,204**	**374**

All three Newman runs exited non-zero. The totals are execution triage, not final defect counts. Candidate SUT failures were independently retested outside Newman with Node HTTP `fetch`, using a fresh backend and seeded SQLite before every trial. Only five defects reproduced in 2/2 trials and were retained:

Verified bug	Pipeline	Severity	Evidence
VB-01 Plaintext password storage	Register	Critical	Raw requests, responses, and DB observations
VB-02 Missing coupon authorization	Coupon	High	Two reset trials
VB-03 Missing product authorization with persistence	Product	Critical	Two reset trials and DB observations
VB-04 Incorrect inclusive minimum	Coupon	High	Two equality-boundary trials
VB-05 Incorrect percent formula	Coupon	Critical	Two exact-calculation trials

The independent verification window was `2026-08-17T22:13:01Z` – `2026-08-17T22:13:06Z`. Fractional rounding, alternative auth/status codes, and unknown-field policy remain specification gaps. Missing expired-token/coupon-state fixtures remain environment defects. Missing DB/UI/storage chains remain test defects. Public product reads and authorized admin actions remain expected behavior.

9. CI/CD

The workflow `hw06-api-tests.yml` runs on relevant push/pull-request paths and manual dispatch. It checks out the repository, initializes only the valid EShop submodule, installs Node `20.20.2`, starts and readiness-checks a newly seeded backend, generates masked runtime credentials, enforces `X-Student-Id: 23127334`, runs a deterministic expected-working gate, and uploads CLI/JUnit/HTML reports plus backend logs for 14 days.

The merge gate deliberately selects one independently audited expected-working case per pipeline: `REG-AI-001`, fixed-discount `CPN-AI-017`, and admin happy-path `PRD-AI-001`. It does not relabel the verified-defect cases as passing. `pipefail` and `PIPESTATUS[0]` preserve Newman failure, while all three suites still execute so evidence can be uploaded.

Passing evidence

- Commit `805c5959dd0a19b3a93035c7829dce595ecf8d40`.
- Run 32076713839, job 95531370911.
- Artifact 9303692181, SHA-256 `C8663684B4AF893F25492D703A32B61B120AF0C6FF9CAEFC2FF206A3E1A52432`.
- Register, Coupon, and Product each executed 26 assertions with 0 failures; job conclusion `success`.
- Real images and artifact inspection: `passing-run.md`.

One-failure demonstration evidence

- Commit `8f7786b96b85233c81e788ac028e5f2c5596f2ef`.
- Run 32078644821.
- Artifact 9304316399, SHA-256 `4299F0DB299EBB157DB97A46AF5FF1D853219892ED4217CA4CA75A7BE5166126`.
- Register remained `26/0`, Product remained `26/0`, and Coupon became `27/1` because of exactly one assertion labelled `CI DEMO FAILURE | intentional single assertion failure`; job conclusion `failure`.
- The demo changed no request, fixture, SUT, real data, or legitimate assertion. Real images and the three-commit history are in `failing-run.md`.

The correct assertion set was restored in commit `5c3074e38c89c99932b80a02cb69df788dce76b3`; run 32079401638, job 95539251820 returned all three suites to `26/0` and uploaded artifact 9304578117.

10. Limitations

1. The full diagnostic collection has 148 primary cases, but the CI merge gate runs only three expected-working paths. A green CI result proves failure propagation and those three paths; it does not prove that all 148 cases pass.
2. Only 31 of 148 full-run cases passed their complete audited oracle. The remaining classifications expose real SUT divergence, incomplete automation, or missing setup; they must not be summarized as a generally passing SUT.
3. The AI set contains 7 `INVALID` and 24 `INCOMPLETE` outputs. Only the 8 incomplete Register rows have workbook corrections; Coupon and Product show zero corrected rows. Unsupported boundaries, rounding assumptions, ambiguous status codes, and unbounded UI assertions remain visible.
4. Coupon disabled/usage-limit/concurrency scenarios need deterministic fixtures, and expired-JWT scenarios need a supplied expired token. Without those preconditions, the 13 environment/setup failures cannot establish a SUT verdict.

5. Sixteen test-script defects identify missing complete chained, multi-iteration, DB, storage, or UI verification. Independent verification repaired only the evidence needed for the five retained bugs; it did not retroactively make every collection assertion complete.
6. SQLite is reset locally per API and in CI per selected suite. The work does not test a production deployment, distributed database, browser rendering pipeline, load behavior, or cross-service concurrency.
7. Fractional discount rounding, duplicate/usage-limit status, 401 versus 403, 400 versus 422, expiration equality, normalization, and unknown-field behavior need specification decisions. Observed implementation behavior is not used to close these gaps.
8. Public CLI/HTML evidence omits sensitive reporter data; machine Newman JSON is Git-ignored because it can contain resolved authentication material. This limits public machine-level replay but prevents credential leakage.
9. GitHub Actions artifacts expire after 14 days. The report retains real run URLs, screenshots, commit SHAs, artifact IDs, and downloaded archive hashes, but long-term artifact availability depends on repository retention.
10. The Postman Console screenshot proves one real GUI request and header log. It does not prove that the Postman GUI Collection Runner executed all folders; the three Newman reports provide that automation evidence.